为更智能的
计算未来
带来全新视角

更互联。
更自适应。
更高效。

SynergeticOS

SYNERGETICS OPERATING SYSTEM

综合文档

协同操作系统 (SOS)

一种基于拓扑、自适应且资源高效的操作系统，
灵感来源于 Buckminster Fuller 的协同几何学。

“整体不仅大于
其部分之和，
而且大于各部分
孤立存在时的
总和。”
— Buckminster Fuller



灵感源自
Buckminster Fuller，
为未来而建。

拓扑
对进程与资源之间的关系进行建模。

自适应调度
响应不断变化的工作负载和系统状况。

资源高效
以更少资源做更多事。

涌现式协同
局部决策，
系统级收益。

作者
Solomon Yaw Adeklo
创始人兼发起人

官方网站
https://synergetic-os.github.io/

GitHub 组织
https://github.com/synergetic-os

协同操作系统（SOS）

对其愿景、原则、架构与研究的全面指南

作者：Solomon Yaw Adeklo

协同操作系统的创始人兼发起人

官方网站：https://synergetic-os.github.io/

GitHub 组织：https://github.com/synergetic-os

创始人简介：https://www.f6s.com/solomon-yaw-adeklo

文档：https://www.researchgate.net/publication/408461895_A_comprehensive_systems_documentation_of_Synergetics_operating_system

对一种受 Buckminster Fuller 协同几何学、系统思维以及“以更少做更多”这一追求所启发的全新计算方法的探索。

前言：为什么这项研究很重要

计算机已成为现代生活中不可或缺的一部分。它们帮助我们沟通交流、管理企业、开展科学研究、设计建筑、教育学生，并发展出曾经难以想象的技术。

然而，每一个计算机应用程序的背后，还有另一类软件，大多数人很少留意：操作系统。

操作系统决定了计算机如何运行程序、分配内存、与硬件通信、管理文件，并协调机器内部同时发生的众多活动。

几十年来，操作系统随着硬件、软件工程、调度算法、安全性和资源管理方面的进步不断演进，造就了强大而可靠的计算平台。

然而，计算技术的持续发展也为我们探索系统架构的其他思路创造了机会。

进程与资源之间的关系能否被更明确地建模？计算机能否针对不断变化的工作负载，更智能地调整其资源分配？几何学、拓扑学、系统科学和自适应协调中的思想，能否启发组织计算的新方式？

这些问题正是协同操作系统（Synergetic Operating System，SOS）的出发点。

SOS 是一项实验性的开源研究计划，探索一种面向拓扑、感知关系的操作系统设计方法。它在思想上受到 Buckminster Fuller 的协同几何学（Synergetics）的启发，也受到他更广泛的理念——通过更好的设计，以更少的资源实现更多的目标——的影响。

本文以技术与非技术读者都能理解的语言，阐述 SOS 的动机、思想基础、拟议架构、研究方法、开发现状以及长期可能性。

贯穿全文的一个重要区分是：**SOS 是一个研究方向，并不是声称已经构建出可以完全替代现有操作系统的、可运行的系统。**其所提出的优势必须通过实现、测试、度量和独立的技术评审来加以验证。

第1章：理解操作系统

1.1 什么是操作系统?

想象一家大型餐厅。

餐厅里有厨师、服务员、顾客、食材、烹饪设备、餐桌，以及有限的厨房空间。所有这些要素必须协同运作。如果每位厨师同时使用同一台烤箱，就会陷入混乱；如果需要时食材短缺，订单就会延误；如果没有人协调工作，餐厅就会变得低效。

餐厅经理帮助协调这些活动，就优先级、可用资源和相互竞争的需求做出决策。

操作系统在计算机内部扮演着类似的协调角色。

计算机拥有处理器、内存、存储和其他硬件。应用程序请求使用这些资源，而且往往是同时请求。操作系统帮助协调这些资源的使用。

例如，当有人一边听音乐、一边浏览网页、一边下载文件时，操作系统帮助这些活动共享计算机的资源。

1.2 操作系统的主要职责

组成部分	简要说明
进程管理	组织当前正在运行的程序。
CPU 调度	决定哪个程序或任务获得处理器时间。
内存管理	分配并保护计算机的工作内存。
文件管理	组织信息的存储与检索方式。
设备管理	协调与键盘、屏幕和网络设备等硬件的通信。
安全与权限	帮助控制哪些程序和用户可以访问特定资源。
用户界面	为人们与计算机交互提供方式。

这些职责相互关联。一个程序可能需要处理器时间、内存、存储访问和网络通信才能完成一项任务。

操作系统必须在保持响应性、可靠性、安全性和公平性的同时，协调这些请求。

1.3 为什么操作系统设计依然重要

不同计算机的工作负载和限制差异很大。手机、科学工作站、云服务器、嵌入式控制器和超级计算机所面临的资源管理问题并不相同。

即使在同一台计算机上，需求也在不断变化。刚才还不重要的任务可能突然变得紧急。程序可能突然请求更多内存，或开始密集计算。

因此，操作系统研究人员持续探索改进调度、资源利用率、安全性、能耗和系统响应性的方法。

SOS 带着一个特定的研究问题进入这一成熟领域：

显式地对工作负载、资源和系统组件之间的关系建模，能否为自适应计算带来新的方法？

这个问题并不预设现有操作系统普遍低效。相反，它探讨的是：另一种架构视角能否为特定类别的工作负载提供有用的思路。

第2章：介绍协同操作系统

2.1 什么是 SOS?

协同操作系统是一项实验性研究计划，探索与协同几何学相关的概念如何为计算机进程和资源的组织与协调提供参考。

其拟议方法强调关系、基于拓扑的表示、资源感知的决策、自适应调度和反馈。

通俗地说，SOS 研究的是：理解计算机系统各部分之间如何相互关联，能否帮助我们更好地决定这些部分应如何协同工作。

来看一个简单的例子。

一台计算机正在运行三项活动：

- 一个需要持续保持响应性的视频会议。
- 一个可以容忍延迟的后台文件传输。
- 一个占用大量处理器时间的数据处理任务。

资源管理系统必须在它们相互竞争的需求之间取得平衡。它可能会考虑优先级、资源可用性、任务之间的依赖关系，以及延迟某项活动所造成的后果。

SOS 探索的是：对这些关系进行更丰富的表示，能否支持更明智的调度决策。

这是一个研究假设。它是否能带来可度量的收益，取决于所评估的模型、算法、实现和工作负载。

2.2 “协同”的含义

“协同”一词在这里指的是对相互作用的组成部分如何影响更大系统行为的关注。

自行车是一个简单的例子。它的车轮、车架、踏板、链条和刹车各有不同的功能，而自行车的整体行为取决于这些部件如何相互连接并协同工作。

改变一个连接或一个部件，就可能影响整辆自行车的行为。

计算机也存在类似的关系。内存分配的变化可能影响处理器的活动；处理器调度决策可能影响应用程序的响应性；一次网络操作可能触发额外的计算。

SOS 研究如何在系统设计中表示这些关系并对其进行推理。

2.3 核心研究方向

该项目汇集了若干相关的思想：

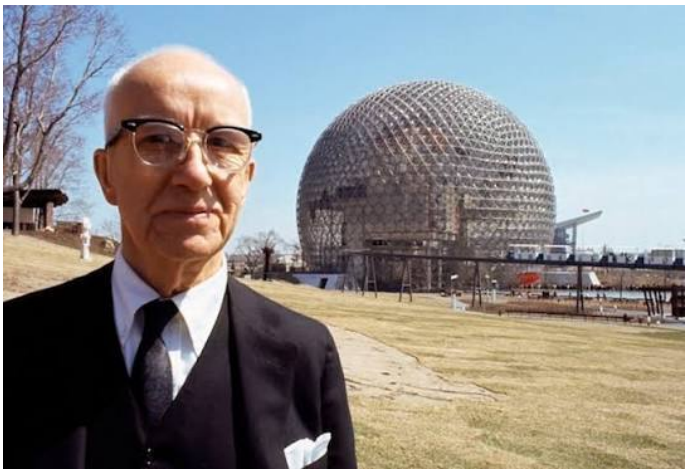
1. **基于拓扑的建模：**表示进程、资源和其他系统元素之间的关系与连接。
2. **资源场抽象：**探索一种模型，以能够为分配决策提供依据的方式表示资源的可用性与需求。
3. **自适应调度：**研究任务调度如何响应不断变化的工作负载和系统状况。
4. **反馈驱动的协调：**利用对系统行为的观察来为后续决策提供依据。
5. **涌现式协调：**研究局部决策与交互能否产生有用的系统级行为。

这些是架构层面的研究概念。其精确的数学定义、实现边界和已证实的性能，必须通过持续开发来确立。

第3章：Buckminster Fuller 与 SOS 的思想基础

3.1 Buckminster Fuller 是谁?

Buckminster Fuller（1895–1983），通称 R. Buckminster Fuller，是美国发明家、建筑师、设计师和系统思想家。他的工作探索了几何学、工程、技术以及对相互关联系统的理解如何为人类环境的设



计提供参考。

他与测地线穹顶、综合设计方法、非物质化（ephemeralization）概念以及两卷本著作《协同几何学》（Synergetics）联系在一起。

Fuller 的兴趣不止于设计单个物体。他经常思考设计决策如何影响更大的系统，包括材料、能源和技术资源的使用。

Buckminster Fuller Institute 将他的设计科学方法描述为发现并应用通用原理来解决实际问题的努力。https://www.bfi.org/about-fuller/big-ideas/design-science/

3.2 什么是协同几何学?

Fuller 的《协同几何学》（Synergetics）发展出一套几何与概念框架，用于探索关系、结构和系统的行为。

该著作把几何学看作不仅仅是绘制形状的方式，而是把空间关系和结构组织视为理解系统如何排布的重要研究对象。

Fuller 的思想包括对四面体及其他几何关系、结构效率，以及各组成部分如何参与更大构型的研究。

他的工作内容广泛，不应被简化为某个口号或一堆几何形状。它代表了一种更广泛的尝试，即考察系统的结构与关系如何影响该系统所能做的事情。

Buckminster Fuller Institute 提供了对其工作的介绍，并附有已出版著作的链接。https://www.bfi.org/about-fuller/big-ideas/synergetics/

3.3 什么是非物质化?

与 Fuller 相关的另一个重要思想是**非物质化**（ephemeralization），通常概括为“以更少做更多”。

这一原则鼓励设计者从有限的材料、能源、时间和其他资源中追求更大的有用性能。Buckminster Fuller Institute 将其视为他的设计科学哲学的核心主题。https://www.bfi.org/about-fuller/big-ideas/design-science/

例如，设想两台执行相同有用任务的机器。如果其中一台在不影响所需结果的前提下耗能更少，那么它可能代表着更节约资源的设计。

这一原则并不是不惜一切代价地减少资源消耗。有意义的比较必须考虑系统完成了什么、完成得有多好，以及需要哪些资源。

3.4 这些思想如何启发 SOS

Fuller 与 SOS 之间的联系是一种思想上的启发，并不意味着 Fuller 设计了操作系统，或提供了现成的计算机架构。

这项研究借鉴了三个宽泛的主题。

第一，关系很重要。Fuller 的工作引导人们思考组成部分如何在更大结构中相互作用。SOS 将这一问题应用于计算任务、资源和系统状况之间的关系。

第二，结构影响性能。在工程中，组件的排布会影响强度、稳定性和材料需求。SOS 研究计算关系的另类表示方式能否支持有用的调度和资源管理决策。

第三，应当检验资源效率。非物质化启发人们思考：系统能否以更少的处理器时间、能耗、内存压力或协调开销来交付所需的结果。

这些联系把一种宽泛的设计哲学转化为可以用计算机科学来研究的问题。

3.5 从哲学启发到可度量的研究

科学工作的一条重要原则是：仅有灵感并不能确立结论。

几何模型并不会自动比传统算法更快。去中心化的安排并不会自动比集中式更高效。受 Fuller 启发的设计也不会继承他工程成就中已被证实的特性。

对于 SOS，相关问题必须直接加以检验：

- 所提出的模型是否有助于调度器做出更好的决策？
- 它是否改善了特定工作负载的响应性？
- 它是否降低了能耗或资源争用？
- 该模型引入了多少计算开销？
- 随着进程数量增加，它是否依然可靠？

这些问题为从概念性的灵感走向基于证据的评估提供了路径。

第4章：为什么要探索另一种操作系统架构?

4.1 相互竞争的需求带来的挑战

一台计算机通常同时运行许多活动。有些需要即时响应，有些则可以等待。有些占用大量内存，有些则受限于处理器速度或网络访问。

系统必须平衡这些需求。

改善某一项指标的调度策略可能会使另一项指标恶化。给某个任务分配更多处理器时间可能缩短其完成时间，但会延迟其他任务。降低能耗可能影响性能。收集更多关于工作负载的信息可能改善决策，但会带来开销。

不存在适用于所有工作负载和所有目标的通用最优策略。

这也是替代性调度模型值得继续研究的原因之一。

4.2 超越孤立的度量

传统的调度决策可能使用任务优先级、执行时间、就绪状态或资源可用性等因素。现代操作系统也可以使用复杂的依赖信息、反馈以及针对特定工作负载的优化。

SOS 研究的是：关于任务与资源之间关系的显式模型，能否提供一个额外的视角。

例如，两个任务可能争用同一资源；另一个任务可能依赖其中一个任务的输出。一次调度决策不仅会影响当前被选中的任务，也会影响相关任务的进展。

基于拓扑的表示可以使这些关系更容易表达和分析。

然而，只有当表示更多关系所带来的决策改进足以抵消增加的计算和工程成本时，这样做才有价值。

4.3 研究机会

SOS 旨在研究一种把系统组织、关系和适应性视为核心架构关注点的设计方法。

其潜在贡献可能包括：

- 表示任务与资源关系的新方法。
- 以这些关系为依据的替代性调度算法。
- 表达资源需求与可用性的模型。
- 响应系统状况变化的反馈机制。
- 评估系统整体权衡的实验方法。

其目标并不是假定成熟的操作系统应被抛弃，而是确定 SOS 的特定概念能否带来可度量的收益，以及这些概念可能在哪些场景中发挥作用。

第5章：SOS 与传统操作系统有何不同

5.1 设计侧重点的比较

传统操作系统的构建方式并不都相同。Linux、Windows、macOS、实时操作系统和嵌入式系统采用不同的架构和调度策略。

许多系统已经支持复杂的资源管理、自适应行为、依赖跟踪和并行执行。

因此，下表比较的是 **SOS 所提出的研究侧重点**，并不是声称每一种传统操作系统都缺乏这些能力。

设计问题	常见操作系统做法	SOS 的研究侧重点
任务如何表示?	进程、线程及其他执行抽象，视系统而定。	探索显式的、感知关系的任务与工作负载表示。
关系如何处理?	通过依赖、同步、优先级和资源所有权等机制。	研究基于拓扑的模型，以表达系统元素之间的关系。
资源如何分配?	通过成熟的分配策略、调度器、内存管理器以及子系统专用机制。	探索资源场抽象及其在协调决策中可能发挥的作用。
调度如何适应?	现有系统使用多种调度策略和反馈机制。	研究由不断变化的关系和资源状况所驱动的调度策略。
如何评估效率?	取决于系统目标、工作负载和实测性能。	检验所提出的架构能否在既定条件下改善选定的效率指标。

5.2 SOS 更高效吗?

这是一个重要的问题，但需要谨慎作答。

SOS 旨在探索可能更具适应性、更节约资源的计算方式。目前它尚未证明自己总体上比现有操作系统更高效。

要可信地声称效率更优，需要针对合适的基线进行可重复的度量。

例如，若声称某个 SOS 调度器改善了任务完成时间，就应使用相同的工作负载、硬件假设和度量流程，将其与传统调度算法进行对比测试。

结果不仅应报告改进，还应报告劣势、开销，以及这种替代方案表现不佳的情形。

5.3 效率意味着什么

效率并不是单一的度量。

一个系统可能耗能更少，但完成任务耗时更长；另一个系统可能完成得很快，但消耗更多内存；某个调度器可能改善了平均性能，却使最慢的任务等待过久。

因此，有用的评估应考虑多项指标。

指标	**说明**
任务完成时间	任务完成所需的时长。
吞吐量	系统在一段时间内完成的工作量。
CPU 利用率	处理器容量被使用的程度。
内存使用量	系统消耗的工作内存量。
单任务能耗	在可度量时，完成既定工作量所需的能量。
调度开销	用于做出调度决策的计算量。
响应时间波动	系统响应时间的一致性。
公平性	各任务是否获得了合适的资源份额。
可扩展性	随着工作负载规模或复杂度增加，系统行为如何变化。

这些度量使我们能够在计算环境中研究非物质化原则。

问题就变得可度量：一个系统能否在不产生不可接受的权衡的前提下，用更少的资源完成相同的必要工作，或用相同的资源完成更多有用的工作？

第6章：SOS 的拟议架构

6.1 概述

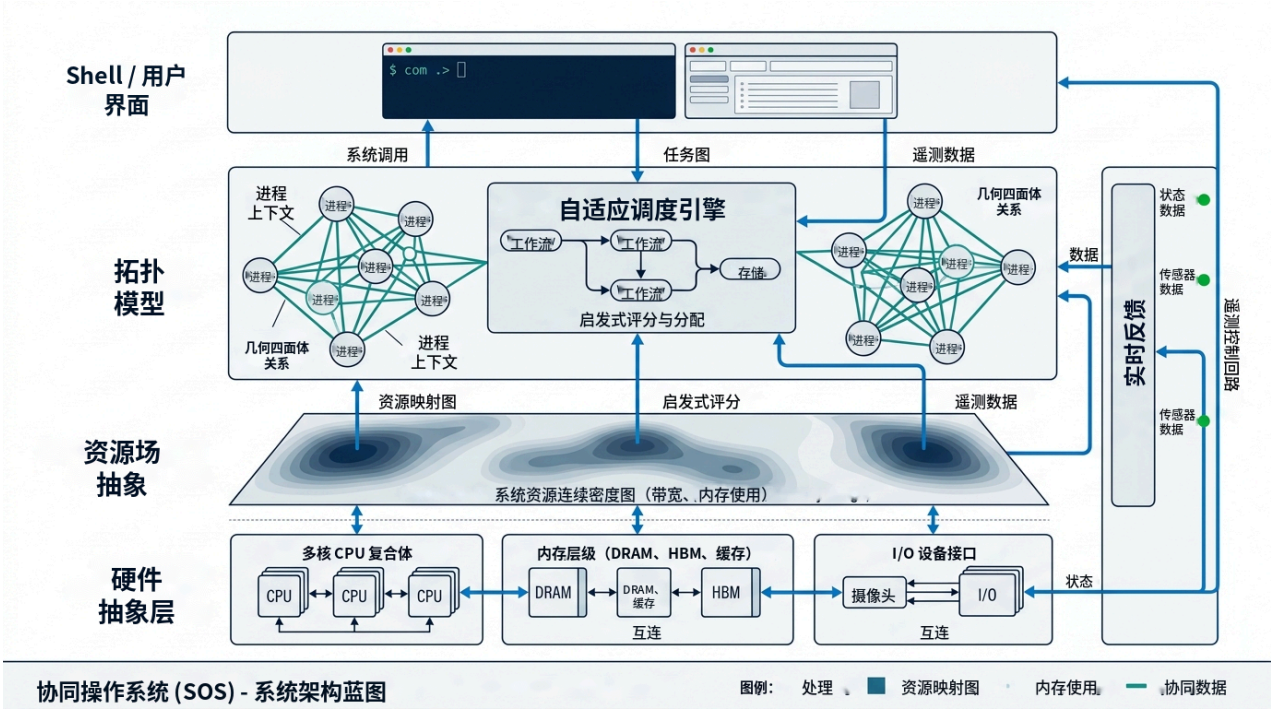
SOS 研究架构可以通过六个概念组件来描述：

1. 进程与工作负载模型。
2. 拓扑表示。

3. 资源场抽象。
4. 自适应调度器。
5. 反馈与评估。
6. 用户界面与系统 Shell。

这些组件是在概念层面上描述预期的架构。不应将其解释为每个组件都已作为可运行的操作系统子系统实现的证明。

6.2 架构图



下图展示了信息与决策的拟议流向。

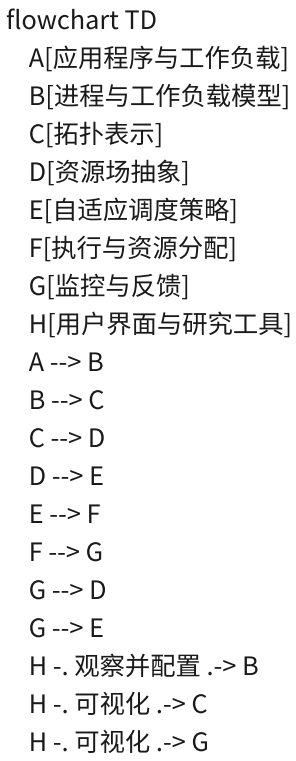


图 1. SOS 概念架构。该图呈现的是一个研究模型，并非一个完整操作系统的、经过验证的实现映射。

6.3 组件一：进程与工作负载模型

进程是一个正在运行的程序，而工作负载是系统必须处理的任务与需求的集合。

拟议的工作负载模型将描述任务的相关属性，例如：

- 它们所需的资源。
- 它们的优先级或时间要求。
- 它们对其他任务的依赖。
- 它们当前的执行状态。
- 随条件变化而变化的资源需求。

以视频编辑器为例。导入素材、生成预览、处理特效和导出成品视频，可能涉及具有不同优先级和资源需求的不同任务。

工作负载模型可以帮助显式地表示这些差异。

具体存储哪些信息、如何收集，取决于具体实现。

6.4 组件二：拓扑表示

拓扑关注关系与连通性。在 SOS 中，基于拓扑的模型可以表示任务、资源和依赖之间的连接方式。

一种实用的表示是图。

图包含**节点**和**边**：节点代表实体，边代表它们之间的关系。

例如，一个节点可以代表一个任务，一条边表示一个任务依赖于另一个任务。

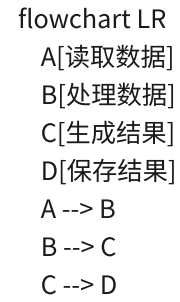


图 2. 一个简单的任务依赖图。

在这个例子中，只有在所需数据可用之后才能开始处理，并且只有在结果生成之后才能保存。

图使依赖结构变得可见。

SOS 可以研究更丰富的图或其他拓扑表示能否帮助调度决策考虑这些关系。

图并不会自动带来更好的性能。其有用性取决于它所表示的信息以及解释它的算法。

6.5 组件三：资源场抽象

资源场抽象是一种拟议的方式，用来表示整个系统中资源的可用性、需求和分布。

想象一个共享工作空间，多个人需要办公桌、工具和电力。工作空间管理者必须了解哪里有可用资源、哪些需求紧急，以及哪里正在产生竞争。

资源场模型旨在提供一种结构化的方式，来表达计算机系统中类似的状况。

相关资源可能包括处理器时间、内存容量、存储访问或其他可度量的系统资源。

例如，模型可以描述：

- 当前可用的处理器容量。
- 活动任务的内存需求。
- 哪些任务在争用共享资源。
- 资源压力是在增加还是在减少。

- 在系统约束下有哪些分配选项可用。

资源场的数学含义必须在实现中精确定义。除非模型明确建立了有意义的数学对应关系，否则不应将其与物理力场混为一谈。

6.6 组件四：自适应调度器

调度器负责决定哪个符合条件的任务应当运行以及何时运行。

自适应调度器研究调度决策是否应当根据观察到的状况而变化。

例如，系统可以在对时间敏感的任务的截止时间临近时给予其更多关注，同时让后台任务使用其他可用资源。

许多现有操作系统已经实现了复杂的调度策略。因此，SOS 需要确立其自身的调度模型在这些成熟方法之外还有哪些贡献。

一个有用的研究过程是：

1. 定义调度问题。
2. 说明拟议的决策规则。
3. 在受控仿真中实现这些规则。
4. 与基线算法比较结果。
5. 找出改进与权衡。
6. 改进模型并重复实验。

6.7 组件五：反馈与评估

反馈机制观察系统行为，并提供可为后续决策提供依据的信息。

对于 SOS，相关观察可能包括任务完成时间、资源利用率、队列长度、内存压力和调度开销。

模型可以利用这些信息评估其决策是否产生了预期的结果。

反馈必须经过精心设计。收集度量会消耗资源，而对暂时性变化反应过度可能导致不稳定的行为。

因此，研究不仅应考察反馈是否改善决策，还应考察其开销和稳定性特征是否可以接受。

6.8 组件六：用户界面与系统 Shell

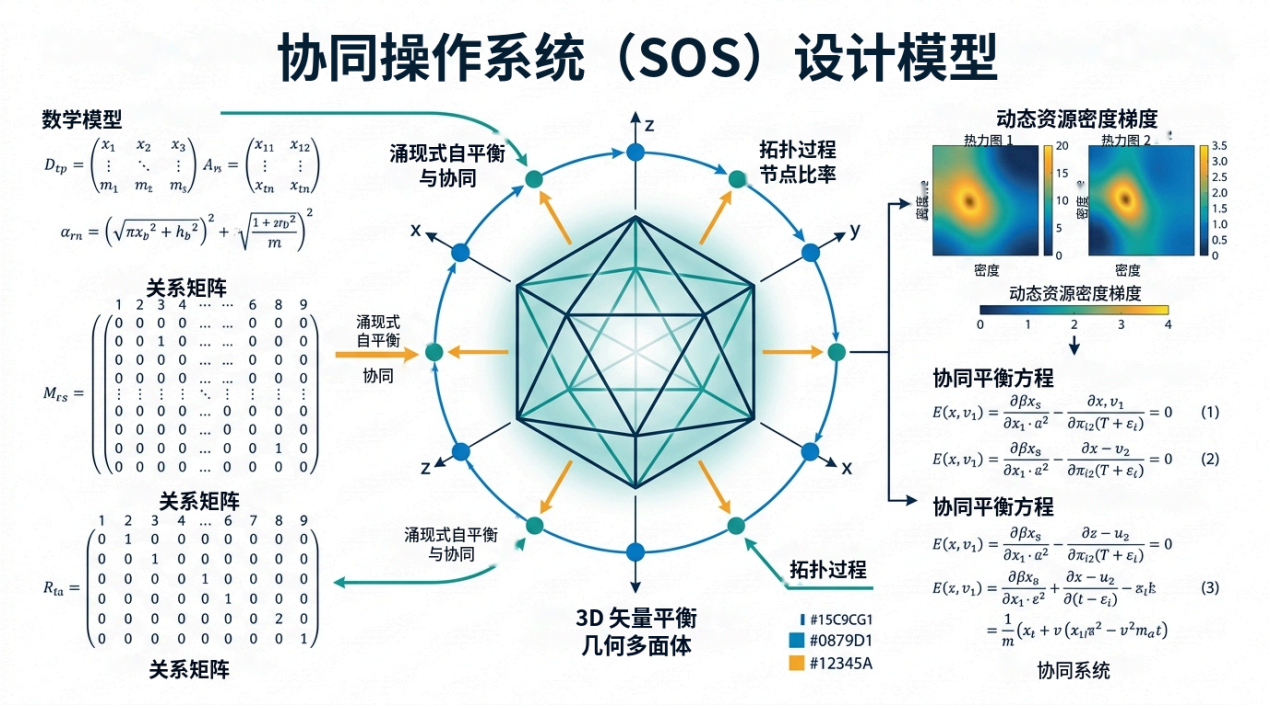
系统 Shell 或图形界面为人们与系统交互提供方式。

对于 SOS，拟议的界面可以帮助开发者和研究人员观察：

- 活动任务及其关系。
- 资源需求与可用性。
- 调度决策。
- 工作负载状况的变化。
- 实验结果和性能指标。

界面主要是使系统行为可观察、可管理的一种方式。一个外观精美的仪表盘本身并不能证明底层架构高效或可运行。

第7章：示意性设计模型



7.1 从孤立的任务到相互连接的任务

传统的任务列表可能显示若干正在运行的程序：

- 浏览器。
- 视频编辑器。
- 文件传输。
- 数据处理程序。

感知关系的表示还可以显示哪些任务依赖于其他任务、哪些在争用资源，以及哪些可以独立推进。

例如，数据处理程序可能需要等待文件传输完成后，才能分析下载的文件。

理解这种依赖关系，有助于避免调度尚无法取得进展的工作。

重要的研究问题是：在计及维护该表示的成本之后，这种额外的表示能否带来比更简单模型更好的决策。

7.2 拟议的调度循环

```
flowchart TD
  A[观察工作负载]
  B[更新任务与资源模型]
  C[评估符合条件的任务]
  D[选择调度决策]
  E[执行任务]
  F[度量结果]
  G[将结果与目标比较]
  A --> B --> C --> D --> E --> F --> G
  G --> A
```

图 3. 示意性的基于反馈的调度循环。

该图展示的是一般性的研究工作流程，并不是声称每一步都已在 SOS 原型中实现。

7.3 设计原则：关系应当服务于决策

拟议的架构不应仅仅为了增加复杂性而收集关系。

只有当关系模型能帮助系统做出本来难以做出、成本高昂或准确性较低的有用决策时，它才有价值。

例如，如果简单的优先级队列对某一特定工作负载始终能产生更好的结果，那么对于该工作负载，也许就没有理由采用更复杂的拓扑模型。

一个可信的 SOS 实现必须愿意发现自身方法在哪些场景有用，以及在哪些场景更适合采用更简单的技术。

第8章：研究、仿真与原型开发

8.1 为什么从仿真开始?

构建一个完整的操作系统是一项庞大的工程，涉及硬件初始化、内存管理、进程隔离、设备驱动、安全、存储、调度以及许多其他组件。

因此，应分阶段地研究架构思想。

仿真提供了一种方式，使我们无需先实现完整的操作系统，就能测试选定的算法。

调度器仿真可以对任务、资源需求、调度规则和任务完成情况进行建模。研究人员随后可以在受控条件下比较不同的策略。

对于 SOS，基于 Rust 的调度器仿真是探索调度概念并收集证据的一条实用研究路径。

调度器仿真不等同于在可启动操作系统内部运行的内核调度器。仿真结果只能证明所建模实验所支持的内容。

调度器仿真的源代码：https://github.com/synergeticos/Simulation/blob/main/synergetic_scheduler_rust_simulation.rs

8.2 为什么选择 Rust?

Rust 是一种强调内存安全、性能以及对重要编程行为进行显式控制的编程语言。

它被用于系统编程，是对注重性能的调度模型进行实验的合理选择。

使用 Rust 并不会自动使某个算法更快，也不能证明系统是安全的。设计、实现、测试和验证的质量依然至关重要。

8.3 拟议的实验方法

可重复的 SOS 评估可以遵循以下流程：

1. **定义问题。**明确工作负载和资源管理问题。
2. **选择基线算法。**选取适合比较的成熟调度方法。
3. **定义 SOS 模型。**记录拟议的拓扑、资源表示和决策规则。
4. **创建受控工作负载。**在相关时包括 CPU 密集型任务、资源争用、依赖关系和不同的任务优先级。
5. **运行重复实验。**使用可比较的配置和足够的重复次数。
6. **度量性能。**记录完成时间、吞吐量、调度开销、资源使用及其他相关指标。
7. **公布方法。**公开假设、源代码和结果以供审阅。
8. **研究局限性。**报告 SOS 表现更差、引入开销或无法扩展的情形。

8.4 公平比较的示例

假设将拟议的 SOS 调度算法与一种传统调度策略进行比较。

公平的实验应使用相同的任务集、硬件或仿真假设、资源限制和终止条件。

实验可以度量：

- 平均任务完成时间。
- 最坏情况或高百分位完成时间。
- 总调度开销。
- 吞吐量。
- 调度模型所占用的内存。

结果应以数字形式报告，并清楚记录实验条件。

在度量结果提供支持之前，不应声称有任何性能提升。

第9章：设计与用户体验

9.1 为什么要将系统可视化?

当复杂系统的内部关系只能通过日志或原始数值输出看到时，很难理解它们。

可视化界面可以帮助研究人员看到任务之间如何相互依赖、哪里的资源承受压力，以及调度决策如何影响系统行为。

对于 SOS，有两个概念性的界面方向尤为相关：

- **桌面或系统 Shell 概念：**用于与系统交互并访问工具的通用环境。
- **拓扑与资源监控概念：**任务关系、资源状况和调度活动的可视化表示。

这些是设计概念。文档中出现它们，不应被解释为这些界面已与可运行的 SOS 内核连接的证明。

9.2 建议的概念性界面图像

以下项目图像可在准备文档时用作视觉参考：

拓扑与资源监控概念

桌面界面概念

项目标志

这些插图传达的是项目预期的设计方向。除非它们准确描绘了已实现并经过测试的界面，否则应将其标注为概念设计。

第10章：创始人与项目起源

10.1 Solomon Yaw Adeklo

协同操作系统由 Solomon Yaw Adeklo 发起，他是一名加纳软件开发者，其在 SOS 上的工作探索系统思维、资源管理与计算架构之间的关系。

该项目源于他对 Buckminster Fuller 协同几何学的兴趣，以及将更广泛的设计原则应用于操作系统研究的可能性。

他的目标是研究：感知关系、面向拓扑的方法能否为建模和协调计算工作提供新的方式。

创始人简介：https://www.f6s.com/solomon-yaw-adeklo

10.2 从构想到研究项目

操作系统概念的开发需要的不只是最初的构想。它涉及阐明问题、定义架构、建立数学模型、实现算法、检验假设并记录结果。

SOS 通过研究文档、仿真、原型开发与协作来遵循这一过程。

该项目还使用包括 ChatGPT 在内的 AI 辅助工具，来支持探索想法、完善文档、思考架构和协助编写代码等任务。这类辅助可以加快部分工作，但不能取代技术专业知识、测试、代码审查或独立验证。

10.3 开放协作的重要性

操作系统开发涉及多个学科。取得进展可能需要在 Rust、内核开发、调度理论、图算法、资源管理、测试、基准测试和技术写作方面有经验的人员的贡献。

开源研究项目可以为这些贡献的讨论、测试和改进提供一个平台。

SOS 欢迎建设性的审视。对其假设、实现和实验结果提出的问题很有价值，因为它们有助于把有前景的概念与尚未证实的主张区分开来。

第11章：当前范围与开发路线图

11.1 为什么必须明确项目状态

研究项目可能包含几种不同类型的工作：已发表的概念、仿真代码、早期原型脚手架、界面设计和未来的实现目标。

这些并不可互换。

已发表的架构并不证明其组件已被实现。仿真并不证明操作系统能够启动。原型并不证明已达到生产就绪。

因此，SOS 项目应分别说明各个组件的状态。

11.2 开发阶段

阶段	主要目标	所需证据
概念研究	定义问题、原则和架构。	清晰的文档、定义和研究问题。
调度器仿真	在受控条件下测试调度模型。	可重复的代码、工作负载、指标和结果。
原型开发	实现选定的组件和界面。	可运行的代码、测试以及记录在案的局限性。
对比评估	与合适的基线比较该方法。	重复实验和已发表的分析。
更深入的系统集成	研究与操作系统和硬件机制的集成。	可运行的组件、集成测试和安全分析。
长期操作系统研究	探索该架构能否支持更完整的系统。	与所声称范围相称的、可演示的启动、执行、资源管理和可靠性能力。

该表是一个路线图框架。各里程碑的状态应根据实际的代码仓库和经过验证的测试结果进行更新。

11.3 成功会是什么样子

成功不应仅仅以拥有一个新的操作系统名称或视觉上独特的架构来定义。

一项有意义的研究贡献可以是：针对明确定义的一类工作负载表现更好的调度算法；使重要关系更易于推理的资源模型；或对某种有用权衡的可重复演示。

表明所提出的方法对特定工作负载没有改善的结果同样可能有价值。它有助于确立模型的边界，并指导未来的工作。

第12章：挑战与开放性研究问题

有若干问题需要认真研究。

计算开销：维护拓扑与资源模型的成本是否超过了它所带来的收益？

可扩展性：随着任务和关系数量增长，所提出的表示能否保持实用？

稳定性：基于反馈的决策能否避免振荡、饥饿和不可预测的行为？

公平性：调度器能否在竞争的任务之间取得平衡，而不会系统性地使某些工作负载处于劣势？

安全性：该架构如何保持应用程序之间的隔离、权限和保护？

可靠性：当度量不完整、资源需求意外变化或某个组件发生故障时，系统应如何表现？

集成：所提出的模型如何与成熟的硬件、驱动程序、内存管理和内核机制交互？

证据：在什么实验条件下，SOS 方法能提供可度量的优势？在哪些情形下不能？

这些问题应被视为研究重点，而不是假定的解决方案。

第13章：SOS 与更广泛计算社区的关系

SOS 最好被理解为操作系统架构与资源管理这一更广泛领域中的一项探索。

它不需要证明每一个现有系统都存在根本缺陷，才能证明其研究问题的合理性。

成熟的操作系统凝聚了数十年的工程与研究成果。它们的调度算法、安全机制和资源管理器，为评估新思想提供了重要的基线。

SOS 可以通过明确定义其模型、实现选定的思想、开展公平的比较，并公布正面与负面的发现，做出有意义的贡献。

项目对 Fuller 的借鉴增添了独特的设计视角，但其技术贡献最终必须通过计算机科学与工程证据来评估。

第14章：协作与参与

项目欢迎有兴趣探索替代性计算方法的人士参与贡献。

可能的贡献领域包括：

- Rust 编程与代码审查。
- 操作系统架构。
- 调度算法与图论。

- 数学建模与优化。
- 资源管理与性能度量。
- 测试、基准测试与可重复性。
- 文档、研究与文献综述。
- 界面设计与系统可视化。

贡献者可以通过审阅假设、改进定义、构建小型实验组件、测试算法，或使项目更易于理解来提供帮助。

如需表达参与意向，请联系：

邮箱：synergeticsoperatingsystem@gmail.com

官方网站：https://synergetic-os.github.io/

GitHub 组织：https://github.com/synergetic-os

创始人简介：https://www.f6s.com/solomon-yaw-adeklo

项目应维护清晰的贡献指南、适当的许可、透明的署名，以及审阅和接受工作成果的书面流程。

第15章：结论——探索一种思考计算的不同方式

协同操作系统始于一个问题：用另一种方式来表示任务、资源和系统组件之间的关系，能否带来有用的计算新方法？

Buckminster Fuller 的协同几何学为考察关系与系统结构提供了思想启发。他的非物质化概念鼓励设计者研究如何以更少的资源实现有用的成果。

SOS 把这些宽泛的思想转化为一个研究方向，涉及基于拓扑的建模、资源场抽象、自适应调度和反馈驱动的协调。

其机遇并不在于假定所提出的架构已经更为优越，而在于构建模型、开发算法、与成熟方法对比测试，并公布证据，让他人能够评估结果。

项目的长期价值将取决于其研究的质量、实现的清晰度、实验的可重复性，以及更广泛计算社区的贡献。

一种新的操作系统设计方法，只有当它的思想能够被检验、实现、测试、质疑和改进时，才变得有意义。

SOS 的雄心是探索：一种更显式地感知关系的计算方法，能否帮助系统更智能地使用其资源。研究仍在进行，其结论必须以证据来赢得。

参考文献与延伸阅读

1. 协同操作系统 — 官方网站
   https://synergetic-os.github.io/
2. 协同操作系统 — GitHub 组织
   https://github.com/synergetic-os
3. Solomon Yaw Adeklo — F6S 个人主页
   https://www.f6s.com/solomon-yaw-adeklo
4. Buckminster Fuller Institute — 协同几何学
   https://www.bfi.org/about-fuller/big-ideas/synergetics/
5. Buckminster Fuller Institute — 设计科学
   https://www.bfi.org/about-fuller/big-ideas/design-science/
6. Fuller 的协同几何学 — 在线参考
   http://www.rwgrayprojects.com/synergetics/intro/explicit.html
7. SOS 研究文章 — DEV Community
   https://dev.to/synergeticos/rethinking-the-operating-system-why-we-need-to-explore-a-synergetic-approach-to-computing-507g
8. SOS 研究文章 — Medium
   https://sadeklo.medium.com/rethinking-the-operating-system-why-we-need-to-explore-a-synergetic-approach-to-computing-22320491b17c

文档状态：研究概览与概念架构。实现方面的声明、基准测试结果和里程碑状态，应随着经过验证的项目证据的出现而更新。

建议的下一步

上述文本为第一份完整草稿。若要将其转化为正式的研究文档，下一阶段应加入实际的 SOS 架构和设计图、插入生成的封面图像，并直接对照项目代码仓库核实当前的实现状态。本草稿中的图表被明确标注为概念性的，以免读者将其误认为是一个完整操作系统的证明。